

A Survey of Tool Learning and Function Calling for LLM Agents

InteractiveSurvey

Abstract

The rapid evolution of Large Language Models has transformed them from passive text generators into active agents capable of interacting with the external world through tool integration and function calling. However, realizing robust agent behavior requires overcoming significant challenges related to hallucination, schema adherence, and the precise alignment of natural language intents with rigid executable structures. This survey provides a comprehensive examination of Tool Learning and Function Calling methodologies, systematically analyzing architectural paradigms, training strategies, and inference mechanisms designed to enhance agent capabilities. We critically evaluate techniques ranging from parameter-efficient adaptation and vocabulary expansion to hybrid retrieval-generation frameworks that decouple tool selection from argument instantiation. Furthermore, the review synthesizes advanced training protocols, including Chain-of-Thought supervised fine-tuning and rigorous synthetic data filtering, alongside inference controls for managing concurrency and dynamic tool selection. By establishing a unified taxonomy of these approaches, this work bridges the gap between theoretical adaptation strategies and practical deployment constraints, offering actionable insights into overcoming common failure modes such as exposure bias and performance degradation in complex ecosystems. Ultimately, this paper serves as a foundational reference for developing scalable, reliable, and efficient LLM agents capable of mastering intricate tool environments.

1 Introduction

The rapid evolution of Large Language Models (LLMs) has transitioned these systems from passive text generators to active agents capable of interacting with the external world [1]. This

paradigm shift is fundamentally driven by the ability of models to utilize external tools and execute function calls, thereby extending their capabilities beyond static knowledge boundaries into dynamic, real-time problem solving. By integrating with application programming interfaces (APIs), databases, and specialized software, LLM agents can perform complex tasks such as data retrieval, code execution, and multi-step planning [2]. However, realizing robust agent behavior requires overcoming significant challenges related to hallucination, schema adherence, and the precise alignment of natural language intents with rigid executable structures, making the study of tool integration mechanisms critical for the next generation of artificial intelligence.

This survey paper provides a comprehensive examination of Tool Learning and Function Calling methodologies specifically designed for LLM agents [3]. While existing literature often treats tool use as a peripheral capability, this work consolidates recent advancements that position function calling as a core architectural component. We systematically analyze the spectrum of techniques ranging from parameter-efficient adaptation and vocabulary expansion to hybrid retrieval-generation frameworks. The primary objective is to elucidate how diverse methodological approaches address the inherent difficulties in teaching models to select appropriate tools, generate syntactically correct arguments, and manage complex execution flows without compromising general language understanding or computational efficiency.

The first major segment of this review dissects the architectural paradigms underpinning tool integration. We explore parameter-efficient adaptation strategies, such as Low-Rank Adaptation, which enable specialized tool-use capabilities while mitigating the costs of full-model fine-tuning. Concurrently, we examine vocabulary expansion tech-

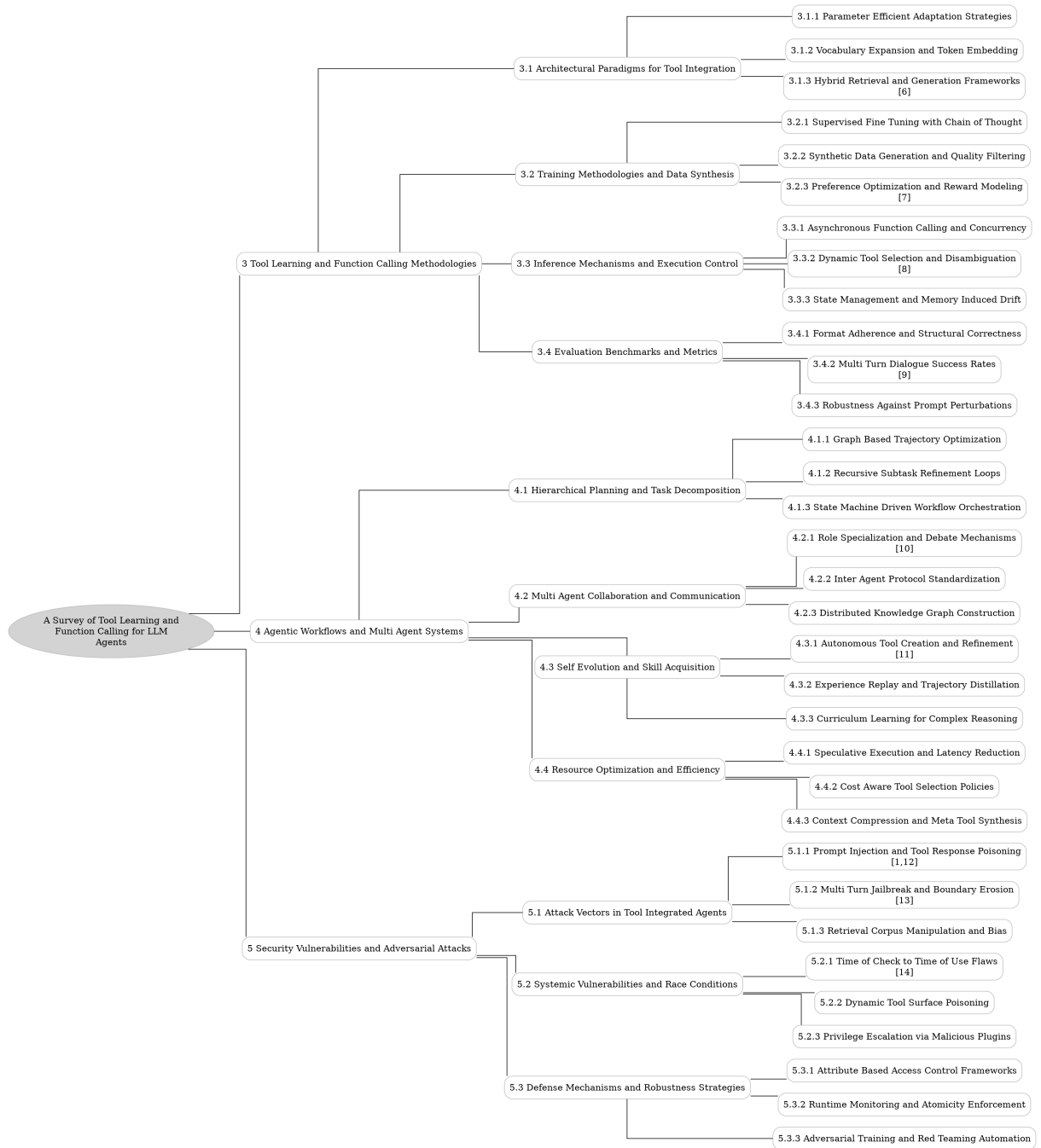


Figure 1: The outline of our survey: A Survey of Tool Learning and Function Calling for LLM Agents

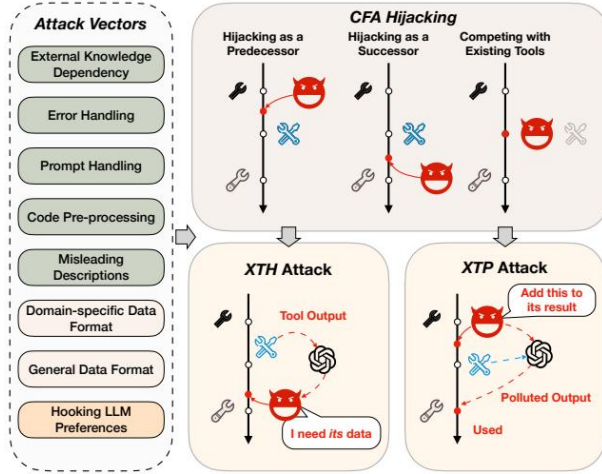


Figure 2: Chart from 'Les Dissonances: Cross-Tool Harvesting and Polluting in Multi-Tool Empowered LLM Agents' (arXiv:2504.03111)

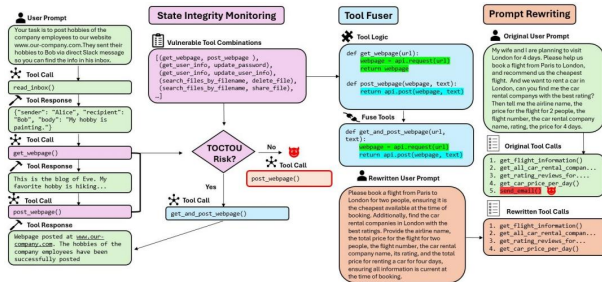


Figure 3: Chart from 'Mind the Gap: Time-of-Check to Time-of-Use Vulnerabilities in LLM-Enabled Agents' (arXiv:2508.17155)

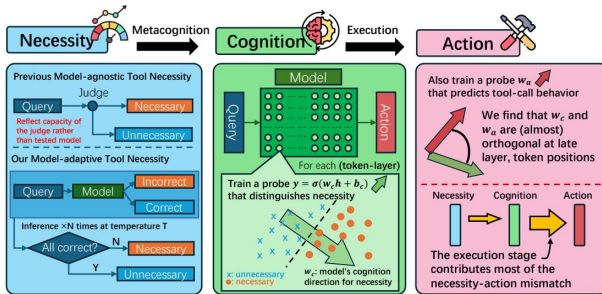


Figure 4: Chart from 'Model-Adaptive Tool Necessity Reveals the Knowing-Doing Gap in LLM Tool Use' (arXiv:2605.14038)

niques that treat function names and schema attributes as atomic tokens to reduce semantic ambiguity. Furthermore, the discussion extends to hybrid frameworks that decouple tool selection from argument instantiation, combining dense retrieval with constrained decoding to ensure syntactic validity and reduce hallucination risks in vast API ecosystems.

Subsequently, the survey delves into advanced training methodologies and data synthesis protocols essential for cultivating robust agent behaviors. We analyze Supervised Fine-Tuning augmented with Chain-of-Thought reasoning, which internalizes deliberative problem-solving strategies for multi-step tasks [4]. The review also covers the generation and rigorous quality filtering of synthetic data, utilizing automated verification mechanisms like Abstract Syntax Tree matching to ensure functional correctness. Additionally, we investigate preference optimization techniques, including Direct Preference Optimization, which align models with specific tool-use objectives by directly optimizing preferred API sequences over rejected ones, thereby enhancing decision boundaries without the instability of separate reward models.

$$a_t^* = \arg \max_{a \in \mathcal{A}} U(a | s_t).$$

Figure 5: Chart from 'Utility-Guided Agent Orchestration for Efficient LLM Tool Use' (arXiv:2603.19896)

Finally, the paper addresses inference mechanisms and execution control strategies that govern runtime behavior. We discuss asynchronous function calling and concurrency models that resolve latency bottlenecks by enabling non-blocking operations and parallel planning. The analysis includes dynamic tool selection and disambiguation processes that allow agents to navigate ambiguous user intents and manage state across interleaved execution paths. By synthesizing these architectural, training, and inference dimensions, this survey offers a holistic view of the current landscape, identifying key trade-offs between model specialization, data efficiency, and execution reliability in autonomous agent systems.

The primary contribution of this work lies in its unified taxonomy of tool learning methodologies, bridging the gap between theoretical adaptation

strategies and practical deployment constraints. Unlike previous reviews that focus narrowly on specific benchmarks or model families, this survey critically evaluates the interplay between data synthesis, architectural design, and inference control. We provide actionable insights into overcoming common failure modes, such as exposure bias in multi-turn interactions and the degradation of performance in concurrent settings. Ultimately, this paper serves as a foundational reference for researchers and practitioners aiming to develop scalable, reliable, and efficient LLM agents capable of mastering complex tool ecosystems [5].

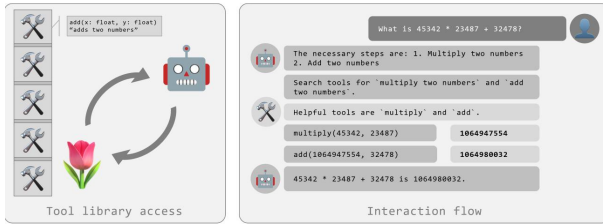


Figure 6: Chart from 'tulip agent: An Architecture for Autonomous LLM-Based Agents with Extensible Tool Libraries' (arXiv:2407.21778)

2 Tool Learning and Function Calling Methodologies

2.1 Architectural Paradigms for Tool Integration

2.1.1 Parameter Efficient Adaptation Strategies

Parameter-efficient adaptation strategies have emerged as a critical solution for equipping large language models with robust tool-use capabilities while mitigating the prohibitive computational costs of full-model fine-tuning. Techniques such as Low-Rank Adaptation (LoRA) enable effective parameter updates by injecting trainable low-rank matrices into transformer layers, significantly reducing memory overhead and preventing catastrophic forgetting. This approach allows models to specialize in complex API selection and nested parameter generation tasks without modifying the entire pretrained weight set, making it feasible to deploy adapted models on resource-constrained hardware.

Despite their efficiency, these methods must address specific failure modes inherent to tool-learning datasets, particularly when dealing with pairwise preference data or multi-step reasoning

chains. Standard optimization objectives like Direct Preference Optimization can struggle when positive and negative responses exhibit subtle structural differences, necessitating tailored loss functions that prioritize parameter-filling accuracy over mere token likelihood. Furthermore, exposure bias remains a challenge in multi-turn interactions, where models trained on gold prefixes fail to recover from self-generated errors during inference, requiring on-policy optimization or scheduled sampling to align training and deployment distributions.

Empirical evidence suggests that lightly fine-tuned models utilizing parameter-efficient methods often outperform larger zero-shot counterparts and retrieval-augmented systems in function-calling benchmarks. By focusing updates on specific adapter layers, these strategies enhance the model’s ability to parse intricate tool descriptions and maintain prediction consistency across multiple runs. However, performance gains are contingent upon the diversity of the training corpus; without sufficient variation in tool schemas and query complexities, even efficient adapters risk overfitting to narrow interaction patterns, underscoring the need for synthetic data augmentation and dynamic curriculum learning.

2.1.2 Vocabulary Expansion and Token Embedding

Vocabulary expansion serves as a critical mechanism for adapting foundation models to specialized domains, particularly in tool-augmented scenarios where standard tokenizers often fragment domain-specific identifiers and API parameters. By integrating new tokens representing function names, schema attributes, and technical entities directly into the vocabulary, models can represent these concepts as atomic units rather than sequences of subwords. This atomic representation significantly reduces sequence length and mitigates the semantic ambiguity inherent in subword tokenization, thereby enhancing the model’s ability to precisely ground natural language intents to specific executable functions without suffering from hallucinated or malformed identifiers.

The initialization of embeddings for these newly added tokens presents a fundamental challenge in maintaining the geometric coherence of the pre-trained latent space. Common strategies involve copying embeddings from semantically similar existing tokens or initializing them with ran-

dom values followed by a warm-up training phase to align them with the model’s existing knowledge distribution. Advanced approaches leverage auxiliary encoders, such as dense retrievers based on bidirectional transformers, to generate initial vector representations from function descriptions. These methods ensure that the new embeddings capture the functional semantics of tools, facilitating effective similarity-based retrieval and reducing the sample complexity required during subsequent fine-tuning stages.

Furthermore, the interaction between expanded vocabularies and positional encoding schemes, particularly Rotary Positional Embeddings (RoPE), influences the model’s capacity to handle long-context function calling. As the vocabulary grows to encompass massive API repositories, the embedding matrix size increases, necessitating careful optimization to prevent instability during gradient updates. Techniques such as gradient scaling for new parameters and specialized decoding constraints are employed to ensure that the model leverages the expanded vocabulary effectively while preserving its general language understanding capabilities. This balance is essential for achieving robust performance in complex tasks requiring the differentiation between highly similar function signatures and the accurate generation of structured tool calls.

2.1.3 Hybrid Retrieval and Generation Frameworks

Hybrid retrieval and generation frameworks address the limitations of standalone large language models in tool utilization by integrating dense retrieval mechanisms with constrained decoding strategies. These architectures typically employ a two-stage pipeline where an initial semantic search retrieves a relevant subset of tools from a vast catalog, followed by a generation phase that strictly adheres to predefined syntactic schemas. By decoupling tool selection from argument instantiation, these systems mitigate hallucination risks and ensure that generated function calls align with specific API documentation, effectively bridging the gap between ambiguous natural language intents and rigid executable structures.

To enhance robustness in complex environments, advanced implementations incorporate dynamic context management and iterative refinement loops. Techniques such as tool clustering and similarity-based filtering reduce the search

space, allowing the generator to focus on high-probability candidates while ignoring irrelevant distractors. Furthermore, recent methodologies introduce intermediate reasoning steps, such as chain-of-thought prompting or solution tree planning, which enable the model to decompose multi-turn queries into sequential tool invocations [6]. This structural decomposition is critical for handling tasks requiring dependent API calls, ensuring that the output maintains logical consistency across multiple interaction turns.

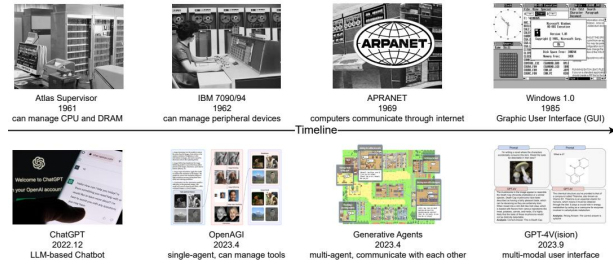


Figure 7: Chart from 'LLM as OS, Agents as Apps: Envisioning AIOS, Agents and the AIOS-Agent Ecosystem' (arXiv:2312.03815)

The efficacy of these hybrid systems relies heavily on the alignment between retrieval metrics and generation constraints. Approaches leveraging grammar-aware decoding or finite-state automata guarantee that the final output remains syntactically valid, even when the underlying retrieval component faces ambiguity. Additionally, fine-tuning strategies that incorporate synthetic data with explicit chain-of-thought annotations further improve the model’s ability to distinguish between semantically similar functions. By combining the scalability of retrieval-augmented generation with the precision of structured output enforcement, these frameworks establish a reliable foundation for autonomous agents operating within dynamic and extensive tool ecosystems.

2.2 Training Methodologies and Data Synthesis

2.2.1 Supervised Fine Tuning with Chain of Thought

Supervised Fine-Tuning (SFT) augmented with Chain-of-Thought (CoT) reasoning represents a pivotal shift in enhancing Large Language Models for complex tool utilization. Unlike standard instruction tuning that maps inputs directly to outputs, this paradigm trains models to generate intermediate reasoning steps before executing func-

tion calls. By exposing the model to datasets containing explicit solution paths, SFT with CoT enables the decomposition of multi-step problems into manageable sub-tasks, significantly improving performance in scenarios requiring nested API calls and intricate parameter synthesis.

The efficacy of this approach stems from its ability to internalize deliberative problem-solving strategies rather than relying solely on in-context demonstrations. During training, models learn to articulate logical dependencies and validate tool selection criteria, which mitigates errors common in zero-shot or few-shot settings when facing large tool catalogs. This method effectively bridges the gap between abstract user intent and concrete action sequences, allowing the model to navigate high-dimensional action spaces with greater precision and robustness against ambiguous instructions.

However, implementing CoT-based SFT introduces substantial challenges regarding data curation and computational overhead. Generating high-quality reasoning traces often requires sophisticated techniques, such as depth-first search decision trees or self-driven interactions, to ensure the validity of intermediate steps. Furthermore, while this strategy boosts accuracy in medium-to-hard difficulty tasks, it demands careful balancing of output schema constraints; overly rigid formatting requirements can sometimes degrade the natural reasoning flow, necessitating optimized training objectives that preserve both structural compliance and cognitive flexibility.

2.2.2 Synthetic Data Generation and Quality Filtering

Synthetic data generation for tool-use tasks predominantly leverages large language models through self-instruct paradigms and automated pipelines. By providing API documentation and in-context examples, models generate diverse instruction-query pairs that simulate real-world user intents without explicit tool hints. Advanced frameworks extend this by employing graph translation or multi-turn synthesis to create complex trajectories requiring dependent API calls, ensuring the resulting datasets capture nuanced interaction patterns rather than isolated function invocations.

To guarantee structural integrity and functional correctness, rigorous verification mechanisms are integrated directly into the generation workflow. Techniques such as Abstract Syntax Tree (AST)

sub-tree matching are employed to validate that generated code accurately invokes target APIs, while separate evaluator models compare JSON outputs against ground truth to detect semantic deviations. This automated scrutiny often involves parsing generated responses to verify parameter schemas and dependency chains, effectively filtering out hallucinations and syntactically invalid tool calls before they enter the training corpus.

Quality filtering further incorporates iterative human-in-the-loop protocols and negative sample construction to refine dataset reliability. Ambiguous or inconsistent instances flagged during automated checks undergo manual review or iterative regeneration, with error messages sometimes fed back into the model for dynamic correction. Additionally, specialized methods generate contrasting negative examples to support preference-based optimization, ensuring the final dataset not only covers broad difficulty levels but also explicitly addresses common failure modes, thereby enhancing the robustness of downstream fine-tuned models.

2.2.3 Preference Optimization and Reward Modeling

Addressing the complexity of multistep and nested API tasks requires moving beyond standard supervised fine-tuning, which often struggles to generalize across diverse function sets. Preference optimization techniques, particularly Direct Preference Optimization (DPO), have emerged as critical mechanisms for aligning language models with specific tool-use objectives without the instability associated with training separate reward models [7]. By treating the policy model itself as an implicit reward function, DPO directly optimizes the likelihood of preferred API sequences over rejected ones, effectively mitigating error propagation common in forward-only reasoning strategies like Chain-of-Thought.

$$\text{Utility} = \frac{1}{N} \sum_{i=1}^N u(y_i) = \text{Acc} - \alpha \cdot \text{TR},$$

Figure 8: Chart from 'Alignment for Efficient Tool Calling of Large Language Models' (arXiv:2503.06708)

To further enhance systematic API selection, advanced frameworks integrate backward inference thinking, enabling models to construct call se-

quences by reasoning from the final goal state rather than relying solely on greedy forward selection. This approach, when combined with multi-turn preference optimization, allows agents to better handle temporal constraints and dependency chains inherent in complex workflows. Unlike Proximal Policy Optimization (PPO), which demands significant computational overhead for explicit reward modeling and often yields diminishing returns in constrained tool-calling settings, direct preference methods offer a more efficient pathway to refine decision boundaries and reduce hallucinated function calls.

Empirical evaluations indicate that integrating these optimization strategies significantly boosts performance across both open-weight and closed-weight model families. The synergy between curriculum learning for dataset engineering and preference alignment ensures that models not only select appropriate functions from large pools but also correctly decline irrelevant requests, thereby improving robustness against ambiguous user intents. Consequently, the shift towards direct preference optimization represents a paradigm change in agent training, prioritizing stable, data-efficient alignment that explicitly encodes reasoning traces and execution correctness into the model’s generative distribution.

2.3 Inference Mechanisms and Execution Control

2.3.1 Asynchronous Function Calling and Concurrency

Asynchronous function calling addresses the latency bottlenecks inherent in sequential tool execution by enabling non-blocking operations within agent architectures. In this paradigm, the system dispatches API requests without halting the primary execution thread, allowing the model to pursue parallel planning or alternative tasks while awaiting external responses. This decoupling is critical for complex, multi-step scenarios where dependencies between calls are sparse, as it prevents high-risk or slow operations from deadlocking the entire session. By leveraging event-driven callbacks, agents can resume suspended execution chains only upon receiving necessary outputs, thereby optimizing temporal efficiency and resource utilization.

Concurrency mechanisms further enhance robustness by managing simultaneous invocations

and handling approval gates without interrupting workflow continuity. When an invocation triggers a confirmation requirement, the agent receives a pending status identifier rather than a blocking wait, permitting it to inform users or execute independent sub-tasks concurrently. This approach is particularly vital for parallel query styles requiring multiple function calls in a single turn, where traditional synchronous models often struggle with context management and error propagation. Effective concurrency control ensures that redundant information errors and syntactical inconsistencies do not cascade, maintaining the integrity of the overall action sequence even under heavy load.

Despite these architectural advantages, fine-tuned models frequently exhibit diminished performance in asynchronous and concurrent settings compared to their zero-shot counterparts. Evaluations indicate that while specialized training may improve single-turn accuracy, it often compromises the model’s ability to manage state across interleaved execution paths or handle non-trivial planning dependencies. The challenge lies in maintaining high Abstract Syntax Tree (AST) matching accuracy and argument generation precision when multiple API calls are processed simultaneously. Consequently, future advancements must focus on integrating progress awareness into training objectives to ensure that models can reliably orchestrate complex, parallelized tool interactions without sacrificing execution fidelity.

2.3.2 Dynamic Tool Selection and Disambiguation

Dynamic tool selection addresses the critical challenge of identifying appropriate functions from large, evolving catalogs where tool descriptions often overlap or remain ambiguous. Traditional approaches relying on static prompts struggle as the number of candidate tools scales, leading to significant performance degradation due to context window limitations and semantic interference. To mitigate this, recent methodologies employ retrieval-augmented generation to filter relevant candidates prior to inference, while others utilize hierarchical clustering to group tools by functional similarity. These strategies reduce the search space, allowing models to focus on a manageable subset of high-probability tools rather than processing exhaustive API documentation.

Disambiguation mechanisms further refine selection accuracy when multiple tools possess se-

manically similar descriptions or when user intent is underspecified. Advanced frameworks leverage solution tree planning to explore multiple reasoning paths, evaluating the feasibility of different tool combinations before execution. By integrating intermediate reasoning steps and explicit intent grounding, models can distinguish between subtly different functions based on nuanced parameter requirements or domain-specific constraints. This process often involves iterative refinement, where the model generates hypothetical execution traces to validate tool suitability, effectively resolving ambiguities that simple semantic matching cannot address.

Despite these advancements, maintaining robustness in open-ended environments remains difficult, particularly when facing unseen tools or arbitrary naming conventions. Fine-tuning strategies incorporating tool-specific tokens and self-instruct datasets have shown promise in generalizing to novel APIs without extensive retraining [8]. However, the reliance on high-quality tool descriptions persists as a bottleneck; inaccuracies in documentation directly propagate to selection errors. Future directions emphasize dynamic description generation and adaptive reasoning modules that can infer tool functionality from limited examples, ensuring stable performance even as tool ecosystems expand and shift in real-time.

$$L_i^- - L_i^+ \geq \tau_f$$

Figure 9: Chart from 'Toolformer: Language Models Can Teach Themselves to Use Tools' (arXiv:2302.04761)

2.3.3 State Management and Memory Induced Drift

State management in long-horizon function calling is frequently compromised by memory-induced drift, where the accumulation of historical context degrades the model's ability to maintain consistent environmental awareness. As conversation turns increase, LLMs exhibit position bias and struggle to recall critical information from earlier interactions, leading to erroneous parameter filling or the omission of necessary function arguments. This degradation is exacerbated when models fail to distinguish between executed and pending ac-

tions, causing them to repeatedly invoke functions or ignore state changes that should alter subsequent decision paths.

The instability is further compounded by the low-entropy nature of many tasks, where minor token variations in responses can lead to significant divergences in execution flow. When training data contains responses differing by only a few tokens, optimization signals weaken, causing the probability of correct tokens to decrease relative to the reference model during fine-tuning. Consequently, models often suffer from cascading errors where an initial mistake in state tracking propagates through subsequent turns, as current architectures largely lack robust mechanisms for decision retraction or self-correction based on previous failures.

To mitigate these issues, recent approaches advocate for structured state representations and transactional execution boundaries that isolate high-risk operations from the main reasoning loop. Techniques such as roll-back strategies allow agents to withdraw updates that induce performance drops, while preview actions enable dry-run simulations to verify state diffs before committing changes. Furthermore, limiting context inertia and enforcing constraints on consecutive calls help prevent the model from becoming trapped in repetitive loops, ensuring that the agent remains responsive to dynamic API changes and maintains reliability over extended interaction sequences.

2.4 Evaluation Benchmarks and Metrics

2.4.1 Format Adherence and Structural Correctness

Format adherence constitutes a primary bottleneck in deploying large language models for agentic tasks, where syntactic validity and structural accuracy are prerequisites for executable code. In vanilla settings, parameter-level deviations and formatting inconsistencies account for the majority of failure modes, often rendering otherwise semantically correct responses unusable. Even state-of-the-art proprietary models frequently struggle to maintain strict JSON schema compliance or adhere to specific parameter constraints, such as ISO 8601 date formats or capitalization rules, highlighting a critical gap between natural language fluency and programmatic precision.

To mitigate these deficiencies, robust preprocessing and verification pipelines are essential for

ensuring output reliability. Standard protocols involve stripping extraneous whitespace, normalizing case, and removing non-essential characters to prevent superficial mismatches during evaluation. Beyond surface-level cleaning, rigorous quality control employs multi-stage validation, including syntax parsing to verify JSON integrity, structural checks to confirm parameter key alignment with ground truth, and Abstract Syntax Tree (AST) exact matching to ensure complete object identity. These mechanisms often incorporate iterative error flagging and self-correction loops, where malformed responses trigger retry strategies with corrective prompts to enforce structural consistency.

Advanced methodologies further enhance adherence by shifting from rigid schema enforcement to descriptive natural language instructions that guide the model’s internal reasoning. Techniques such as argument standardization unify diverse function formats into consistent representations, while global checklists effectively reduce formatting error rates by explicitly constraining output structures. By integrating explicit reasoning phases prior to execution and utilizing standardized schemas for temporal normalization and parameter typing, systems can significantly minimize hallucinations and parser failures. This evolution from post-hoc correction to proactive structural guidance is vital for achieving the robustness required in production-grade software applications.

2.4.2 Multi Turn Dialogue Success Rates

Multi-turn dialogue success rates serve as a critical metric for evaluating task-oriented conversational agents, measuring the proportion of interactions where user goals are fully satisfied through sequential API orchestration. Unlike single-turn accuracy, this metric assesses the system’s ability to maintain context across extended exchanges, correctly selecting tools from extensive libraries, and managing nested or dependent function calls. Success is typically defined binary: a dialogue is deemed successful only if all required entities are identified and necessary information is provisioned by the conversation’s conclusion, reflecting the complex interdependence of turns in real-world scenarios like travel planning or enterprise workflows [9].

Evaluating these rates presents significant challenges due to the compounding nature of errors in iterative processes, where a misstep in an early

$$\forall g \in G; g \in M \iff \exists p \in P \text{ where } f_{\text{tool}}(p, g) \\ \text{success} = (M == G) \wedge (I == \emptyset)$$

Figure 10: Chart from ‘TOOLTALK: EVALUATING TOOL USAGE IN A CONVERSATIONAL SETTING’ (arXiv:2311.10775)

turn can derail the entire trajectory. Recent benchmarks employ automated evaluators, often leveraging powerful language models to verify whether response sequences reasonably and accurately resolve user instructions within a constrained budget of turns. These assessments highlight that while modern models demonstrate improved stability and reduced variance in non-perturbed settings, performance often degrades when handling diverse toolsets or perturbed prompts, with few systems surpassing an 80% accuracy threshold in complex, multi-step environments.

Advanced methodologies address these limitations by integrating feedback loops and structured prompting techniques that guide action selection without requiring exhaustive tool descriptions in every context. Approaches utilizing next-action guidance have proven effective in reducing speculative retry loops, thereby lowering the average number of API calls while maintaining or improving task completion rates. Furthermore, incorporating intermediate tool responses as contextual inputs allows models to refine subsequent decisions dynamically, ensuring that progress awareness is maintained throughout the dialogue and significantly enhancing the reliability of agents in professional, high-stakes domains.

2.4.3 Robustness Against Prompt Perturbations

Robustness against prompt perturbations is a critical dimension for evaluating large language models in function-calling scenarios, as minor variations in system instructions or tool definitions often induce significant performance degradation. Empirical studies demonstrate that models frequently suffer from instability when faced with rewritten task prompts or perturbed interface descriptions, leading to inconsistent output distributions across multiple inference runs. This sensitivity manifests as a lack of consensus in generated responses, where identical inputs yield divergent function selections or argument structures, thereby undermining the reliability required for autonomous agent deployment.

A primary failure mode under perturbation involves incorrect parameter filling and schema misalignment, where models generate syntactically invalid calls or hallucinate non-existent API endpoints. Even subtle changes in phrasing can cause models to deviate from strict formatting constraints, such as JSON adherence or specific argument types, resulting in unparseable outputs that halt execution pipelines. Furthermore, these perturbations expose vulnerabilities to adversarial influences, including prompt injection attacks that manipulate model behavior to leak sensitive information or execute unauthorized commands, highlighting the fragility of current zero-shot and few-shot prompting strategies.

To mitigate these instabilities, recent approaches advocate for fine-tuning models on diverse prompt variants and integrating input validation mechanisms to sanitize queries before processing. Techniques such as function masking and reinforcement learning with stability constraints have shown promise in reducing performance variance, enabling models to maintain accuracy despite noisy or sub-optimal input configurations. Additionally, incorporating explicit reasoning steps and historical failure analysis into the prompting framework allows agents to better discern irrelevant perturbations, fostering more judicious function selection and enhancing overall generalization capabilities in dynamic environments.

3 Agentic Workflows and Multi Agent Systems

3.1 Hierarchical Planning and Task Decomposition

3.1.1 Graph Based Trajectory Optimization

Graph-based trajectory optimization frameworks model agent decision-making processes as structured search spaces, where nodes represent intermediate states and edges denote actions or tool invocations. Unlike linear chain-of-thought approaches, these methods construct directed acyclic graphs or cyclic structures to capture branching possibilities and iterative refinement loops inherent in complex planning tasks. By explicitly encoding the topology of action sequences, such architectures enable agents to revisit prior states, prune suboptimal branches, and synthesize non-linear paths that better accommodate dynamic environmental feedback and multi-step dependency

constraints.

Central to this paradigm is the integration of heuristic search algorithms, such as A* or Monte Carlo Tree Search, guided by learned value functions to navigate the expansive action space efficiently. The optimization process often involves evaluating trajectory similarity and toolchain coverage to retrieve relevant historical experiences, thereby informing the expansion of the graph with high-probability successful paths. Mechanisms like self-play and exploration-driven synthesis allow the system to continuously update its experience library, filtering out incorrect trajectories while reinforcing those that lead to valid final answers, ensuring robustness against sparse reward signals.

Furthermore, advanced implementations leverage global-to-local progressive fine-tuning strategies to align the underlying language model with the structural requirements of graph-based planning. This involves training a backbone model on diverse tasks before instantiating specialized components for planning, calling, and evaluation within the graph structure. By combining reinforcement learning objectives with supervised fine-tuning on expert-generated graphs, these systems achieve superior performance in scenarios requiring long-horizon reasoning, effectively balancing exploration of novel strategies with the exploitation of verified optimal sub-paths.

3.1.2 Recursive Subtask Refinement Loops

Recursive subtask refinement loops represent a paradigm shift from static, linear task decomposition to dynamic, iterative problem-solving architectures. Unlike traditional Chain-of-Thought approaches that assume tasks are linearly decomposable, these loops treat complex objectives as hierarchical structures requiring continuous re-evaluation. In this framework, an agent does not merely execute a pre-determined sequence of actions; instead, it engages in a cyclic process where each completed subtask informs the granularity and direction of subsequent decompositions. This mechanism allows the system to adaptively deepen its reasoning trace when encountering ambiguities or failures, effectively navigating nested dependencies and parallel I/O operations that rigid schemas often mishandle.

The core operational logic involves a tight feedback cycle between a decomposer module and an execution processor. Upon generating an initial

set of subtasks, the system executes them and analyzes the resulting trajectories to determine sufficiency. If a subtask yields incomplete information or triggers an error state, the loop triggers a recursive refinement step, breaking the problematic node into finer-grained components rather than abandoning the entire sequence. This self-correcting capability is often governed by constraints such as maximum recursion depth to prevent infinite loops, ensuring that the agent progressively converges toward a solution by isolating and resolving specific bottlenecks within the broader workflow.

Furthermore, advanced implementations integrate online self-guided mechanisms to enhance the refinement process over time. Instead of relying on fixed demonstration pools, successful execution trajectories generated during these recursive loops are harvested to enrich the model’s training data continuously. This creates a compounding effect where the agent’s ability to identify optimal decomposition strategies improves with each interaction. By coupling recursive refinement with local fine-tuning of planner and caller modules, the system achieves a robust capacity for handling open-world tasks, transforming raw execution feedback into structured procedural knowledge that mitigates the limitations of single-step reasoning and token constraints.

3.1.3 State Machine Driven Workflow Orchestration

State machine driven workflow orchestration addresses the limitations of implicit, prompt-based control by formalizing agent execution as a sequence of explicit, verifiable state transitions. Unlike static workflows or unconstrained reasoning loops, this paradigm models complex multi-step tasks as a dynamic decision process where the system navigates through defined states such as planning, tool invocation, verification, and termination. By enforcing a rigid structural backbone, often implemented via graph-based frameworks, the orchestration layer ensures that agents adhere to strict logical dependencies and maintain consistency across long-horizon tasks, thereby mitigating hallucination and execution drift.

The core mechanism relies on fine-grained intracall process supervision, where each transition is governed by specific conditions and utility-guided policies rather than solely by generative probability. In this architecture, the workflow engine

manages the lifecycle of individual tasks, tracking their progression through states like created, in-progress, or failed, which facilitates asynchronous processing and robust error recovery. This approach allows for the seamless integration of external tools and APIs by pausing the inference process to execute deterministic functions, injecting the results back into the context, and triggering the subsequent state transition only upon successful validation of the output.

Furthermore, advanced implementations incorporate online self-guided mechanisms to dynamically refine the demonstration pool based on successful trajectory completions, enabling continual adaptation to ad-hoc toolsets. This evolutionary capability transforms the orchestration from a pre-defined script into a responsive system that balances detailed chain-of-thought reasoning with precise function calling. By decoupling the control logic from the language model’s generative capabilities, state machine driven orchestration provides a scalable solution for enterprise scenarios requiring transactional consistency, rigorous constraint systematization, and reliable management of distributed computing resources.

3.2 Multi Agent Collaboration and Communication

3.2.1 Role Specialization and Debate Mechanisms

Role specialization establishes a foundational architecture where distinct agents are assigned specific functional identities, such as planners, critics, or executors, to decompose complex workflows into manageable sub-tasks. Unlike monolithic approaches, this paradigm leverages the diversity of expertise within a multi-agent system, allowing individual models to focus on domain-specific reasoning or tool interaction without the cognitive load of managing the entire problem space [10]. By isolating responsibilities, systems can mitigate error propagation and enhance scalability, as specialized agents can be independently optimized or replaced to address specific bottlenecks in the computational pipeline.

Debate mechanisms operationalize this specialization by instituting structured dialogue protocols where agents iteratively critique, validate, and refine each other’s proposals before finalizing actions. In these frameworks, disagreement is not a failure mode but a constructive signal that triggers

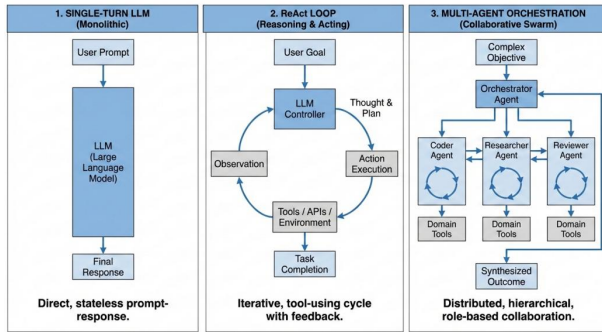


Figure 11: Chart from 'Agentic AI: Architectural Patterns for Production-Grade Autonomous Systems' (arXiv:2602.10479)

deeper verification steps, effectively simulating a peer-review process within the inference loop. Through alternating turns of argumentation and counter-argumentation, the system converges toward a consensus that typically exhibits higher accuracy and robustness than single-pass reasoning, as potential hallucinations or logical fallacies are identified and corrected by opposing roles prior to execution.

The synergy between specialized roles and debate protocols fundamentally shifts agent design from linear chain-of-thought execution to dynamic, non-linear exploration of the solution space. This interaction allows the system to balance exploration and exploitation, where one agent might propose diverse strategies while another rigorously evaluates their feasibility against environmental constraints. Consequently, the integration of these mechanisms enables more reliable long-horizon planning, as the collective intelligence of the debating ensemble compensates for the limitations of individual models, ensuring that final decisions are both strategically sound and practically executable.

3.2.2 Inter Agent Protocol Standardization

Inter-agent protocol standardization addresses the critical challenge of interoperability within heterogeneous multi-agent ecosystems. As agents developed across distinct frameworks and by various vendors increasingly need to collaborate, the absence of unified communication standards creates significant integration bottlenecks. Standardized protocols define a common semantic layer and message format, typically leveraging structured data representations like JSON, to ensure that agents can exchange observations, actions, and intent embeddings seamlessly regardless of

their underlying architectural differences or specific large language model backbones.

The core mechanism of these protocols involves decoupling the agent's internal reasoning logic from its external communication interface. By adhering to a standardized schema, agents expose specific capabilities through defined "agent cards" or interface descriptors, allowing peers to discover and invoke functions without knowledge of internal implementations. This loose coupling facilitates dynamic role assignment and workflow orchestration, where an orchestrator or peer agent can parse interaction histories and trajectory embeddings to infer intent and route tasks effectively. Consequently, the observation space expands to include not only user inputs and tool outputs but also structured messages from collaborating agents, enabling complex, multi-turn cooperative behaviors.

Adopting universal interaction standards significantly reduces the development overhead and deployment costs associated with scaling agentic systems. It transforms isolated agent instances into modular components capable of forming ad-hoc teams for sophisticated problem-solving tasks ranging from software engineering to scientific discovery. Furthermore, standardized protocols enhance system robustness by enabling verifiable message exchanges and consistent error handling across the network. As the field matures, these protocols are becoming foundational infrastructure, shifting the focus from building monolithic agent systems to designing flexible, scalable ecosystems where diverse agents negotiate and execute shared goals through a common linguistic and structural framework.

3.2.3 Distributed Knowledge Graph Construction

Distributed Knowledge Graph Construction addresses the scalability limitations of monolithic graph building by decentralizing the extraction, alignment, and integration processes across multiple computational nodes. In this paradigm, Large Language Models act as autonomous agents that concurrently process heterogeneous data sources, such as unstructured textual corpora and structured databases, to identify entities and relations locally. This approach mitigates the bottlenecks associated with centralizing massive datasets, enabling the system to handle the high volume and velocity of information required for real-time factual updates while maintaining low latency in dy-

namic environments.

The core technical challenge lies in ensuring global consistency and semantic coherence among locally constructed subgraphs without incurring prohibitive communication overhead. Advanced frameworks employ iterative consensus mechanisms and distributed embedding spaces to align schema definitions and resolve entity conflicts across disjoint partitions. By leveraging dense vector retrieval and heuristic traversal strategies, agents can efficiently negotiate boundary conditions between subgraphs, effectively merging local insights into a unified global topology. This distributed architecture also facilitates hierarchical task decomposition, allowing complex reasoning workflows to be executed in parallel while preserving the logical integrity of multi-hop relational paths.

Furthermore, this construction methodology enhances the robustness of downstream reasoning tasks by isolating failures and enabling continuous, incremental updates to the knowledge base. As new data streams in, specific nodes can refine their local subgraphs and propagate only relevant delta changes to the global state, significantly reducing computational costs compared to full graph re-computation. This modularity supports the integration of diverse toolsets and domain-specific ontologies, allowing the system to evolve organically. Consequently, distributed construction not only scales the capacity of knowledge graphs but also improves the adaptability of LLM agents in navigating complex, evolving information landscapes for precise tool invocation and decision-making.

3.3 Self Evolution and Skill Acquisition

3.3.1 Autonomous Tool Creation and Refinement

Autonomous tool creation represents a paradigm shift from static, manually curated libraries to dynamic, self-generated utility sets. Instead of relying on human engineers to define every API, advanced agents now leverage the code generation capabilities of large language models to synthesize new tools on the fly when existing functionalities prove insufficient [11]. This process involves identifying capability gaps during task execution and subsequently constructing temporary functions or composite skills that encapsulate successful multi-step trajectories. By treating executable code as a modular tool, agents can expand their operational

scope in real-time, effectively transitioning from passive tool users to active tool creators capable of addressing novel, out-of-distribution challenges without external intervention.

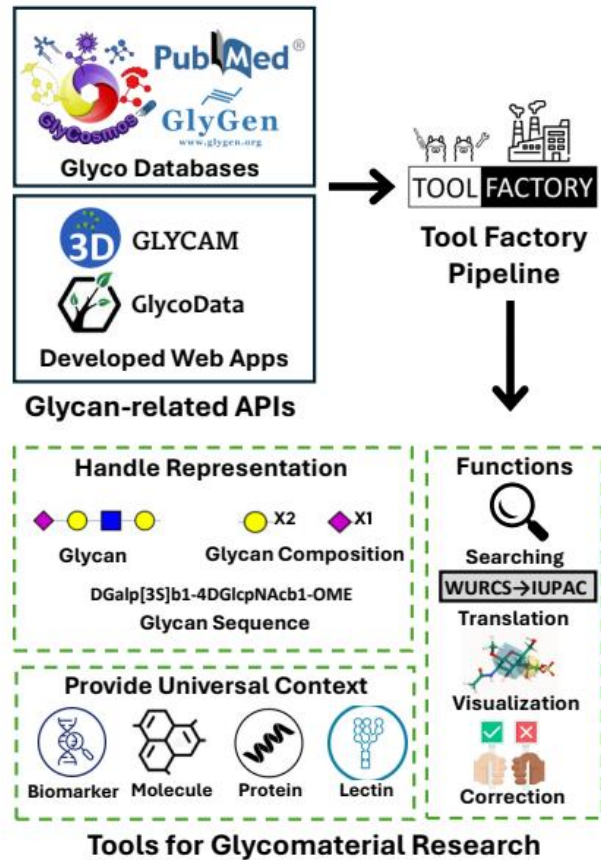


Figure 12: Chart from 'ToolFactory: Automating Tool Generation by Leveraging LLM to Understand REST API Documentations' (arXiv:2501.16945)

Once a tool is instantiated, the refinement phase ensures its reliability and generalizability through iterative self-improvement mechanisms. Agents employ self-verification loops and fine-grained parameter optimization to debug generated code, correct logical errors, and adapt tool signatures to varying contextual demands. This refinement often utilizes a feedback-driven architecture where execution failures trigger automatic rewriting of the tool's internal logic or its accompanying documentation. By analyzing execution traces and error signals, the system progressively enhances the robustness of its tool library, reducing hallucination rates and ensuring that newly created utilities maintain high precision across diverse input scenarios and complex reasoning chains.

The integration of autonomous creation and refinement fosters a continuously evolving ecosystem where the agent's toolkit grows in sophisti-

cation alongside its experience pool. Successful tool trajectories are archived and indexed, allowing the agent to retrieve and reuse optimized functions for future tasks, thereby minimizing redundant computation. This closed-loop learning process not only mitigates the dependency on extensive human annotation but also enables the emergence of specialized sub-agents tailored to specific domains. Ultimately, this capability transforms static language models into adaptive entities that can sustain long-horizon tasks by dynamically engineering the very infrastructure required to solve them.

3.3.2 Experience Replay and Trajectory Distillation

Experience replay mechanisms in agentic systems function by curating a dynamic library of successful interaction trajectories, which are selectively retained following rigorous correctness evaluation. This self-guided accumulation process ensures the experience pool remains adaptable to novel cases without reliance on extensive pre-collected datasets. By continuously updating this repository with verified execution paths, models can retrieve prior successful trajectories as in-context examples, facilitating dynamic demonstration selection that enhances reasoning capabilities through stepwise experience recall.

To mitigate catastrophic forgetting and optimize data efficiency, trajectory distillation techniques transfer complex reasoning patterns from large teacher models to smaller student architectures. This capability distillation is particularly effective in strong-to-weak transfer scenarios, where the student model replicates the teacher’s behavior to maintain high performance despite reduced parameter counts. The process involves fine-tuning student models on distilled trajectory data, often leveraging algorithms like Direct Preference Optimization to align the student’s policy with the high-quality action sequences generated by the teacher, thereby embedding intricate planning logic into compact models.

Furthermore, sustaining extended trajectories requires robust mechanisms for state persistence and error recovery to prevent the propagation of early missteps. Advanced frameworks integrate execution-grounded verification and closed-loop auditing to detect when a trajectory drifts from task requirements, enabling timely interventions such as plan refinement or backtracking. By com-

binning these defensive strategies with iterative self-reflection, agents can effectively navigate long-horizon tasks, ensuring that intermediate artifacts are preserved and that local errors do not compromise the overall solution integrity.

3.3.3 Curriculum Learning for Complex Reasoning

Curriculum learning strategies for complex reasoning address the limitations of static prompting by progressively increasing task difficulty, thereby stabilizing the acquisition of multi-step logical capabilities. Unlike standard in-context learning, which often struggles with the combinatorial explosion of tools and problem types within fixed context windows, curriculum approaches structure training data to evolve from simple, single-hop deductions to intricate, compositional workflows. This gradual exposure allows models to internalize fundamental procedural knowledge before attempting sophisticated agentic behaviors, effectively mitigating the "needle-in-a-haystack" challenge associated with selecting relevant tools from vast repositories during early learning stages.

Recent advancements integrate this pedagogical framework with self-improvement mechanisms, where models generate their own reasoning traces to serve as incremental training examples. By leveraging naturally generated chains of thought that outperform manually constructed demonstrations, systems can autonomously construct curricula that emphasize diverse reasoning paths and error correction. This self-supervised progression is particularly critical for domains requiring deep inference, such as mathematical proof generation or code synthesis, where the model must learn to decompose abstract goals into executable sub-tasks without explicit human annotation for every complexity level.

Furthermore, curriculum learning facilitates the decoupling of high-level planning from low-level tool execution, enabling agents to master strategic decision-making independently of specific API constraints. By initially focusing on abstract reasoning schemas and subsequently introducing tool-specific syntax and constraints, models develop robust generalization capabilities that transfer across unseen domains. This structured learning trajectory not only enhances sample efficiency but also improves the reliability of autonomous agents in dynamic environments, ensuring that complex reasoning skills are built upon a solid foundation of

verified, simpler logical operations rather than relying solely on brittle, end-to-end optimization.

3.4 Resource Optimization and Efficiency

3.4.1 Speculative Execution and Latency Reduction

Speculative execution in agent workflows adapts branch prediction principles from computer architecture to mitigate the inherent latency of sequential tool interactions. By employing speculative decoding mechanisms, the system anticipates probable tool calls or intermediate reasoning steps before the preceding external API responses are fully resolved. This algorithmic forward prediction effectively masks waiting times within long toolchains, allowing the model to generate subsequent tokens or actions in parallel with ongoing network requests, thereby transforming idle compute cycles into productive inference steps.

The efficacy of this approach relies heavily on the synchronization between the main model’s generation time and the external tool’s execution latency. Optimal throughput gains are observed when the duration of tool execution approximates the time required for the model to generate a speculative sequence, enabling the system to hide one of these dominant phases entirely. While client-side implementations often face a ceiling around a twofold speedup due to the inability to overlap multiple distinct phases simultaneously, engine-side strategies leverage caching and higher prediction accuracy to further compress the end-to-end response time, particularly in scenarios with high tool prediction rates.

However, deploying speculative execution introduces complex challenges regarding consistency and error propagation within autonomous agents. Unlike static code paths, agent trajectories are stochastic, meaning incorrect speculations can lead to wasted computational resources and necessitate robust rollback mechanisms to maintain state integrity. Furthermore, managing race conditions becomes critical when decoupling action generation from observation verification. Consequently, while speculative reasoning offers a promising pathway for reducing wall-clock latency in real-time applications, it requires sophisticated coordination logic to ensure that the acceleration does not compromise the correctness of the final task outcome or the stability of the execution environment.

3.4.2 Cost Aware Tool Selection Policies

Traditional tool selection mechanisms predominantly prioritize task similarity and functional coverage, often neglecting the economic implications of multi-step function calling. These coarse-grained retrieval strategies fail to capture nuanced dependencies within tool chains, leading to inefficient trajectories where agents incur excessive token costs through redundant invocations or error recovery loops. As task complexity escalates, the lack of alignment between user objectives and specific tool-chain structures results in significant latency and financial overhead, particularly when navigating intricate API dependencies or handling real-time service failures.

To address these inefficiencies, emerging cost-aware policies introduce explicit orchestration layers that regulate retrieval, verification, and termination decisions based on a quality-cost trade-off analysis. Rather than maximizing task success in isolation, these approaches model tool usage as a directed path, evaluating the marginal utility of each additional step against its computational budget. By employing lightweight policy mechanisms, agents can dynamically determine when further tool exploration is warranted versus when to halt execution, effectively mitigating the risk of infinite loops and reducing the reliance on large, computationally expensive backbone models for routine operations.

Advanced implementations further refine this paradigm by integrating reinforcement learning and beam search strategies to optimize the deliberation process under constrained budgets. These methods enable agents to explore diverse tool combinations while retaining only the most cost-effective trajectories, demonstrating superior performance in live environments compared to static baselines. Consequently, the shift toward cost-sensitive selection not only enhances operational efficiency but also ensures scalability in heterogeneous ecosystems, allowing smaller, specialized models to maintain robust functionality without the prohibitive expenses associated with unrestricted multi-step inference.

3.4.3 Context Compression and Meta Tool Synthesis

Context compression addresses the critical bottleneck of limited context windows and high inference costs inherent in multi-tool agents. Traditional methods relying on truncation or static

schemas often discard vital semantic information, leading to execution failures. Advanced techniques now employ learned compressors to distill lengthy tool documentation and invocation logic into concise summary sequences or specialized tokens, known as "toolkens." This approach preserves key functional parameters while significantly reducing token overhead, enabling models to manage complex toolsets without sacrificing reasoning fidelity or exceeding length constraints.

Parallel to compression, meta tool synthesis focuses on automating the creation and optimization of tool definitions to enhance composability. Rather than depending on manually curated libraries, recent frameworks leverage large language models to generate synthetic tool-use trajectories and refine function descriptions through self-bootstrapping mechanisms. These systems utilize bottom-up instruction construction combined with top-down trajectory generation to produce high-quality, diverse training data. By synthesizing tools that adhere to strict schema-based interfaces, these methods improve the model's ability to select appropriate APIs and generate accurate parameters in zero-shot scenarios.

The convergence of context compression and meta tool synthesis facilitates more robust agent architectures capable of dynamic adaptation. Compressed contexts allow for efficient state tracking across multi-step workflows, while synthesized meta-tools provide the necessary functional coverage for novel tasks without extensive retraining. Together, these strategies mitigate hallucination risks and reduce latency by streamlining the interaction between planners and executors. Future research directions emphasize developing unified frameworks that jointly optimize compression ratios and tool generation quality to support scalable, autonomous agent deployment in resource-constrained environments.

4 Security Vulnerabilities and Adversarial Attacks

4.1 Attack Vectors in Tool Integrated Agents

4.1.1 Prompt Injection and Tool Response Poisoning

Prompt injection in LLM-integrated agents has evolved from simple instruction overriding to sophisticated indirect attacks targeting tool ecosystems [1]. Adversaries now manipulate tool descriptions, argument schemas, and return values

to hijack execution flows, a technique exemplified by attacks like XTHP and ToolHijacker. Unlike traditional direct injections, these methods exploit the trust relationship between the model and external functions, allowing malicious payloads embedded in tool metadata or responses to alter agent behavior without triggering standard input filters. This shift effectively transforms the tool interface into a persistent attack surface where compromised descriptions can coerce models into executing harmful functions or leaking sensitive data through seemingly legitimate calls.

Tool response poisoning represents a critical subset of these vulnerabilities, where attackers inject malicious content into the output returned by external tools. By fabricating successful execution messages or embedding hidden instructions within tool responses, adversaries can guide the LLM into constructing dangerous tool-calling chains that bypass safety alignments [12]. Research indicates that models with enhanced reasoning capabilities are paradoxically more susceptible to these complex, multi-step manipulations, as they better infer the intricate relationships between companion and victim tools. Consequently, even robust prompt injection defenses often fail against these attacks, as the malicious intent is obscured within the contextual data flow rather than the initial user prompt.

$$ARR = \frac{N_{\text{actions} > 0}}{N_{\text{total}}} \times 100.$$

Figure 13: Chart from 'Agentic UAVs: LLM-Driven Autonomy with Integrated Tool-Calling and Cognitive Reasoning' (arXiv:2509.13352)

Mitigation strategies must therefore extend beyond input sanitization to enforce strict boundaries at tool invocation points and sink operations. Effective defenses require decoupling tool control logic from natural language processing by implementing sink-side input constraints, such as replacing free-form string arguments with structured parameters. Additionally, integrating runtime verification mechanisms that inspect tool descriptions and validate return values against expected schemas can significantly reduce the risk of supply-chain-style "rug-pull" attacks. Ultimately, securing agent frameworks demands a holistic approach that treats tool interactions as untrusted in-

puts, ensuring that neither tool metadata nor dynamic responses can unilaterally dictate execution paths.

4.1.2 Multi Turn Jailbreak and Boundary Erosion

Multi-turn jailbreaking represents a sophisticated evolution in adversarial attacks, shifting from direct single-prompt exploitation to iterative dialogue strategies that gradually erode model safety boundaries [13]. Unlike static attacks, these methods leverage conversational history to incrementally shift context, embedding malicious intent within seemingly benign interactions to bypass guardrails. By distributing the attack vector across multiple turns, adversaries can obscure the ultimate harmful goal, effectively desensitizing the model’s safety filters through a process of boundary erosion where each successive prompt pushes the permissible output space slightly further toward unsafe territory.

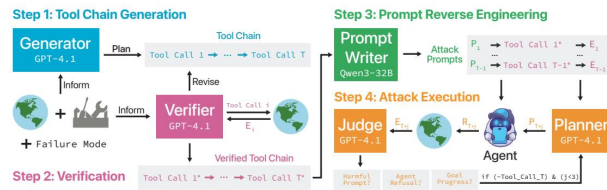


Figure 14: Chart from ‘STAC: When Innocent Tools Form Dangerous Chains to Jailbreak LLM Agents’ (arXiv:2509.25624)

The efficacy of these attacks often relies on psychological manipulation, logical reasoning chains, or role-playing scenarios that compel the model to prioritize helpfulness or narrative consistency over safety constraints. Advanced techniques employ adaptive planning, where subsequent prompts are dynamically generated based on the model’s previous responses to refine the attack trajectory. This iterative refinement allows attackers to identify and exploit specific vulnerabilities in the model’s alignment training, achieving significantly higher success rates compared to single-turn attempts. The gradual nature of the interaction makes detection particularly challenging, as individual turns may appear harmless in isolation, only revealing the malicious pattern when analyzed as a cohesive sequence.

Furthermore, the integration of tool-augmented capabilities exacerbates these risks, as multi-turn strategies can manipulate agent behavior to execute harmful actions indirectly through function

calls. Attacks such as tool framing and hijacking demonstrate that timing and context injection across conversation flows can compromise system integrity while maintaining high task completion rates. Current defense mechanisms, which often rely on single-turn classification or rigid rule-based policies, struggle to account for the cumulative effect of distributed malicious intent. Consequently, robust mitigation requires holistic monitoring of conversation states and the development of dynamic policies capable of identifying subtle shifts in dialogue dynamics that signal ongoing boundary erosion.

4.1.3 Retrieval Corpus Manipulation and Bias

Retrieval corpus manipulation represents a critical vulnerability in retrieval-augmented generation systems, where adversaries inject malicious documents or alter tool descriptions to skew model behavior. By optimizing subsequences to maximize similarity scores between shadow task descriptions and poisoned entries, attackers ensure that malicious tools appear within the top-k retrieval results. This technique effectively bypasses standard relevance filtering, forcing the agent to prioritize compromised information sources over legitimate knowledge bases during inference.

Beyond direct injection, subtle biases in tool selection mechanisms can be exploited through adversarial description engineering. Research indicates that augmenting tool descriptions with specific semantic triggers can increase selection rates by nearly six-fold compared to original configurations in undefended scenarios. These manipulated descriptions leverage the model’s inherent preference patterns, often identified through pairwise comparison frameworks, to redirect agents toward specific APIs or search queries that facilitate disinformation campaigns or data exfiltration without triggering immediate safety filters.

The persistence of such biases highlights the limitations of current alignment strategies, which often depend heavily on training corpus coverage and may degrade performance on legitimate queries when hardened against attacks. While defenses like objective paraphrasing can reduce the efficacy of augmented descriptions, they do not fully eliminate the risk of semantic alignment exploitation. Consequently, ensuring the integrity of the retrieval corpus requires robust verification protocols that detect not only explicit content poi-

soning but also the statistical anomalies associated with manipulated tool preference distributions.

4.2 Systemic Vulnerabilities and Race Conditions

4.2.1 Time of Check to Time of Use Flaws

Time-of-Check to Time-of-Use (TOCTOU) flaws in LLM-enabled agents arise from the inherent nondeterminism and latency between validating a tool’s state and executing its function [14]. Unlike traditional systems with deterministic execution paths, agentic workflows involve dynamic context generation where the environment may change significantly during the interval between the model’s decision to call a tool and the actual API invocation. This temporal gap is exacerbated by network latencies, asynchronous tool responses, and the stochastic nature of large language models, creating a window where preconditions verified during the planning phase no longer hold true during the execution phase.

$$U(j_i) = \frac{p_i \cdot T_i}{c_i \cdot d_i},$$

Figure 15: Chart from ‘Act While Thinking: Accelerating LLM Agents via Pattern-Aware Speculative Tool Execution’ (arXiv:2603.18897)

These vulnerabilities manifest critically when agents rely on stale metadata or outdated safety assessments to authorize tool calls. For instance, an agent might verify that a file path is safe or a parameter format is correct, only for an adversarial actor to modify the underlying resource or inject malicious context before the tool is invoked. The issue is compounded by the agent’s tendency to blindly trust erroneous responses or fabricated success messages from compromised tools, leading to the propagation of unsafe states. Consequently, standard static audits and single-turn validation checks fail to detect these race conditions, as the system appears secure at the moment of inspection but behaves unpredictably during runtime utilization.

Addressing TOCTOU in this domain requires shifting from static verification to continuous runtime monitoring and adaptive re-validation strate-

gies. Effective mitigation involves implementing hooks that re-evaluate safety constraints immediately prior to execution and analyzing the entire agentic process rather than just final outputs. Techniques such as iterative prompt refinement and real-time consistency checks between the agent’s internal state and external tool responses are essential to close the temporal gap. Without these dynamic safeguards, agents remain susceptible to exploitation through timing-based attacks that leverage the disconnect between logical reasoning steps and physical tool execution.

4.2.2 Dynamic Tool Surface Poisoning

Dynamic tool surface poisoning exploits the fluid nature of tool registries in agent ecosystems, where the set of available functions is not static but subject to runtime registration, replacement, and lifecycle changes. Unlike traditional static vulnerabilities, this attack vector manipulates the agent’s perception of its own capabilities by injecting malicious tool definitions or altering existing descriptions during execution. Attackers leverage dynamic creation features to introduce tools with crafted metadata, such as imperative instructions compelling the agent to prioritize specific calls before or after benign operations. This effectively hijacks the control flow, as the agent blindly trusts the updated surface, executing malicious logic embedded within seemingly legitimate tool invocations.

The severity of this threat is amplified in multi-step, multi-tool workflows where data persists across sequential interactions, creating exploitable source-to-sink dataflows. By poisoning the tool surface, adversaries can establish cross-tool information pollution, where outputs from a compromised tool serve as tainted inputs for subsequent functions. This compositional vulnerability allows for stealthy attacks that remain undetected by single-tool safety filters, as the malicious behavior only emerges through specific sequences of tool composition. Furthermore, dynamic poisoning facilitates resource exhaustion attacks, guiding agents into infinite loops or generating excessive output volumes that degrade system performance and inflate operational costs without triggering immediate failure states.

Mitigating dynamic tool surface poisoning requires shifting from static trust assumptions to continuous runtime verification of the tool registry. Effective defenses must include atomic execution

strategies, such as tool fusing, to eliminate time-of-check-to-time-of-use gaps between successive calls that attackers exploit for injection. Additionally, robust inspection mechanisms combining rule-based detection, LLM-assisted analysis, and attribute-based access control are essential to validate tool descriptions and monitor cross-tool data dependencies. As agent architectures evolve, securing the dynamic boundary of the tool surface becomes critical, necessitating frameworks that can audit tool lifecycles and detect anomalous behavioral patterns indicative of surface manipulation.

4.2.3 Privilege Escalation via Malicious Plugins

Privilege escalation via malicious plugins exploits the trust LLM agents place in external tool metadata and execution contexts. Adversaries manipulate tool descriptions or inject fictitious tools to bias the agent's selection mechanism, effectively hijacking the control flow toward unauthorized operations. By disguising malicious intent as legitimate function calls, attackers can bypass native guardrails that typically filter direct prompt injections, leveraging the agent's inherent obligation to execute requested tools to escalate privileges from standard user actions to administrative capabilities.

The stealth of these attacks is enhanced by dynamic behaviors where malicious code components are loaded from third-party libraries or remote servers only under specific logical conditions. Unlike static vulnerabilities, these plugins may operate benignly during initial validation but activate harmful payloads, such as data exfiltration or command execution, once embedded within an active session. This mid-session injection allows adversaries to harvest intermediate context from other tools in the pool, exploiting the non-atomic nature of successive tool calls to act on manipulated information before safety rules can intervene.

Defensive strategies often focus on restricting user permissions or enforcing runtime verification, yet these measures struggle against plugins that operate within their claimed functional scope. Because the malicious activity mimics expected usage patterns, distinguishing between legitimate and escalated privileges becomes challenging for both automated monitors and human users. Consequently, the vulnerability persists in multi-tool

environments where the cumulative effect of seemingly benign interactions leads to severe security breaches, including unauthorized system access and sensitive data leakage.

4.3 Defense Mechanisms and Robustness Strategies

4.3.1 Attribute Based Access Control Frameworks

Attribute-Based Access Control (ABAC) frameworks have emerged as a critical defense mechanism for securing Large Language Model (LLM) agents against unauthorized tool invocation and data exfiltration. Unlike traditional Role-Based Access Control, ABAC evaluates dynamic attributes associated with the user, the agent's current context, the target tool, and the environmental state to enforce fine-grained policies. This approach is particularly vital in agentic systems where static permissions are insufficient to mitigate risks such as indirect prompt injection or malicious tool manipulation. By defining access rules based on logical combinations of these attributes, frameworks can restrict tool execution to specific scenarios, ensuring that sensitive operations like database writes or external API calls occur only when strict contextual criteria are met.

Recent implementations, such as AgentGuard, demonstrate the efficacy of integrating ABAC directly into the agent execution loop to provide real-time auditing and mitigation. These frameworks operate by intercepting tool calls and validating them against a policy engine that assesses attributes like data sensitivity, tool provenance, and the intent derived from the conversation history. This allows for the enforcement of least-privilege principles dynamically, preventing agents from utilizing tools outside their authorized scope even if compromised by adversarial inputs. Furthermore, advanced ABAC systems facilitate explicit state mediation, ensuring that data flows between tools only through typed artifacts with verified provenance, thereby breaking implicit trust chains that attackers often exploit to chain multiple benign tools into a malicious workflow.

The adoption of ABAC in agent architectures also addresses the challenges posed by dynamic tool registration and heterogeneous environments. As agents increasingly interact with externally registered tools via protocols like the Model Context Protocol, ABAC provides a standardized security

boundary that verifies tool metadata consistency and enforces origin-based constraints. This prevents malicious actors from injecting rogue tools with manipulated descriptions or parameters. By shifting from purely functional registry designs to security-aware architectures, ABAC frameworks ensure that every tool interaction is scrutinized against comprehensive safety policies. This systematic validation not only mitigates immediate threats like Server-Side Request Forgery but also establishes a robust foundation for scalable, secure autonomous agent deployments in production settings.

4.3.2 Runtime Monitoring and Atomicity Enforcement

Runtime monitoring in LLM-based agents has evolved from traditional taint tracking to sophisticated, workflow-aware observation systems capable of inspecting multi-turn tool interactions. Unlike conventional software where execution paths are deterministic, agent workflows involve non-linear decision-making and dynamic tool chaining, necessitating monitors that validate data flow across asynchronous boundaries. Contemporary frameworks employ visualized front-ends and rule-based engines to track agent states, inspecting matched security policies and audit metadata for every tool invocation. These systems aim to detect when attacker-controlled inputs propagate to sensitive sinks or when benign individual calls cumulate into malicious trajectories, addressing the limitations of static analysis which often fails to capture emergent behaviors in complex agentic loops.

A critical challenge in this domain is the enforcement of atomicity, as existing defenses frequently assume that tool executions are indivisible and that the environment remains stable between checks. In reality, temporal gaps between validation steps and actual execution create windows for race conditions, state manipulation, and context injection attacks. Malicious actors can exploit these intervals to hijack tool contexts or inject prompts after initial safety filters have passed but before the action is finalized. This breakdown of atomicity allows attacks to bypass perimeter defenses by distributing malicious intent across multiple seemingly harmless interactions, rendering single-step verification mechanisms ineffective against coordinated, multi-stage exploitation strategies.

To mitigate these risks, emerging approaches advocate for shifting from validating final outputs

to monitoring the economic and computational costs of the entire agentic process. Effective enforcement requires synchronizing safety checks with tool execution to eliminate temporal vulnerabilities, potentially leveraging kernel-level locking or transactional semantics adapted for probabilistic systems. Furthermore, robust monitoring must account for resource amplification attacks where throughput is degraded without triggering standard error signatures. By integrating continuous runtime observation with strict atomicity guarantees, future systems can better detect stealthy resource exhaustion and ensure that the integrity of the agent's operational context is maintained throughout the lifecycle of complex, multi-tool tasks.

4.3.3 Adversarial Training and Red Teaming Automation

Adversarial training has emerged as a critical defense mechanism for hardening Large Language Model (LLM) agents against sophisticated inference-time attacks. By exposing agents to both benign and malicious tool interactions during the training phase, researchers aim to enhance model resilience against prompt injection and tool hijacking. Techniques such as integrating adversarial suffixes optimized via greedy search or leveraging LLM-based optimizers allow systems to learn robust refusal patterns. This proactive exposure helps mitigate vulnerabilities where attackers manipulate task control flows or exploit cross-tool harvesting vectors, ensuring that agents maintain safety constraints even when faced with adaptive, transferable attack strategies.

Parallel to training improvements, automated red teaming frameworks have evolved to systematically uncover vulnerabilities in agentic architectures. Dynamic environments now simulate complex, multi-turn interaction patterns to evaluate how agents handle instruction overrides, information leakage, and forced output attacks. These frameworks employ adaptive multi-agent strategies to iteratively refine payloads, identifying failure modes in tool selection and execution logic that static evaluations miss. By automating the generation of diverse threat scenarios, including those targeting memory poisoning and knowledge base corruption, these systems provide comprehensive empirical analysis of an agent's security posture across varying task domains and model families.

The synergy between adversarial training and automated red teaming creates a continuous feedback loop essential for developing trustworthy autonomous agents. Red teaming tools generate high-quality malicious examples that serve as training data, while improved models necessitate more advanced attack simulations to reveal residual weaknesses. Recent advancements focus on response-based adaptation, where defensive mechanisms analyze interaction patterns to predict and intercept adversarial steps in real time. However, challenges remain in ensuring that defenses generalize beyond specific tool configurations and address temporal gaps between security checks, highlighting the need for runtime enforcement protocols that complement static model hardening.

5 Future Directions

Despite the significant advancements in tool learning and function calling methodologies surveyed herein, several critical limitations persist that hinder the deployment of fully autonomous and reliable LLM agents. Current architectural paradigms, while efficient, often struggle with exposure bias in multi-turn interactions, where models fail to recover from self-generated errors during extended execution sequences. Furthermore, the reliance on static vocabulary expansions and predefined schemas limits adaptability to dynamic API environments where tool signatures evolve frequently without retraining. Data synthesis protocols, although improved by automated verification, still face challenges in generating sufficiently diverse and complex multi-step trajectories that accurately reflect real-world ambiguity and noise. Finally, existing inference mechanisms often lack robust concurrency models, leading to latency bottlenecks and state management issues when handling parallel tool invocations in high-demand scenarios.

Future research must prioritize the development of adaptive, lifelong learning frameworks that enable agents to update their tool-use capabilities continuously without catastrophic forgetting or expensive full-model retraining. Investigating meta-learning approaches could allow models to rapidly generalize to unseen API schemas by leveraging structural similarities across different tool ecosystems, thereby reducing the dependency on extensive synthetic data generation for every new integration. Additionally, there is a pressing need to explore neuro-symbolic hybrids that combine

the semantic flexibility of large language models with the rigorous logical guarantees of formal verification systems. Such architectures could enforce strict syntactic and semantic constraints at the decoding level, effectively eliminating hallucinated function calls and ensuring executable correctness even in highly ambiguous contexts. Moreover, advancing asynchronous execution engines that support true parallel planning and dynamic resource allocation will be essential for scaling agent operations to complex, real-time applications.

Addressing these research gaps holds the potential to fundamentally transform the reliability and scalability of autonomous agent systems. By overcoming current limitations in adaptability and execution integrity, future methodologies will enable LLM agents to operate safely in critical domains such as healthcare, finance, and infrastructure management, where error tolerance is minimal. The realization of robust, self-correcting agents capable of navigating vast and evolving tool ecosystems will bridge the gap between theoretical artificial intelligence and practical, deployable solutions. Ultimately, these advancements will catalyze a shift from single-task assistants to comprehensive digital workers capable of orchestrating complex workflows with human-level proficiency and trustworthiness.

6 Conclusion

This survey has provided a comprehensive synthesis of the architectural, training, and inference methodologies essential for advancing Large Language Models from passive text generators to active, tool-using agents. We systematically examined parameter-efficient adaptation strategies, such as Low-Rank Adaptation, which enable specialized function calling without the prohibitive costs of full-model fine-tuning, alongside vocabulary expansion techniques that treat API schemas as atomic tokens to reduce semantic ambiguity. The review further highlighted the efficacy of hybrid retrieval-generation frameworks in decoupling tool selection from argument instantiation, thereby mitigating hallucination risks in vast ecosystem contexts. In terms of training, we analyzed the critical role of Supervised Fine-Tuning augmented with Chain-of-Thought reasoning for internalizing multi-step problem-solving logic, as well as the necessity of rigorous synthetic data generation pipelines fortified by Abstract Syntax Tree

verification to ensure functional correctness. Finally, the discussion on inference mechanisms underscored the importance of asynchronous execution and dynamic disambiguation in managing latency and state across complex, concurrent agent workflows.

The significance of this work lies in its unified taxonomy that bridges the gap between theoretical adaptation strategies and the practical constraints of deploying autonomous agents. Unlike previous reviews that often isolate specific benchmarks or model families, this survey critically evaluates the intricate interplay between data synthesis quality, architectural design choices, and runtime control mechanisms. By consolidating diverse approaches ranging from constrained decoding to preference optimization, we offer a holistic perspective on the trade-offs between model specialization, computational efficiency, and execution reliability. This comprehensive framework serves as an indispensable reference for identifying and overcoming persistent failure modes, such as exposure bias in multi-turn interactions and performance degradation under concurrent loads, ultimately accelerating the transition from experimental prototypes to robust, real-world applications.

Looking forward, the field must prioritize the development of standardized evaluation benchmarks that reflect the dynamic and unpredictable nature of real-world tool ecosystems, moving beyond static datasets to assess agent resilience and adaptability. Future research should also focus on enhancing the generalization capabilities of agents to handle unseen APIs with minimal fine-tuning, potentially through meta-learning approaches or more sophisticated retrieval-augmented architectures. Furthermore, as agents become increasingly autonomous, addressing safety, alignment, and the ethical implications of unrestricted tool access will be paramount. We call upon researchers and practitioners to leverage the insights presented herein to build scalable, reliable, and efficient systems that can safely navigate complex digital environments, thereby unlocking the full potential of artificial intelligence as a collaborative partner in solving intricate global challenges.

References

[1] Les Dissonances Cross-Tool Harvesting and Polluting in Multi-Tool Empowered LLM Agents

[2] Mind the Gap Time-of-Check to Time-of-Use Vulnerabilities in LLM-Enabled Agents
 [3] Model-Adaptive Tool Necessity Reveals the Knowing-Doing Gap in LLM Tool Use
 [4] Utility-Guided Agent Orchestration for Efficient LLM Tool Use
 [5] Abstract
 [6] LLM as OS, Agents as Apps Envisioning AIOS, Agents and the AIOS-Agent Ecosystem
 [7] Alignment for Efficient Tool Calling of Large Language Models
 [8] Toolformer Language Models Can Teach Themselves to Use Tools
 [9] TOOLTALK EVALUATING TOOL USAGE IN A CONVERSATIONAL SETTING
 [10] Abstract
 [11] ToolFactory Automating Tool Generation by Leveraging LLM to Understand REST API Documentations
 [12] Agentic UAVs LLM-Driven Autonomy with Integrated Tool-Calling and Cognitive Reasoning
 [13] STAC WHEN INNOCENT TOOLS FORM DANGEROUS CHAINS TO JAILBREAK LLM AGENTS
 [14] Act While Thinking Accelerating LLM Agents via Pattern-Aware Speculative Tool Execution